Luke Berger (770220)
Benedikt Schröder (773793)
Alexander Roskamp (777951)

Einleitung:

I. **Hintergrund**
Im Rahmen des Praktikums soll ein reduzierter Klon des Puzzlespiels Crazy Machines (2005) entwickelt werden. Ziel ist es, eine Physik basierte Umgebung bereitzustellen, in der KI-Agenten ihre Fähigkeiten im Bereich Physical Reasoning evaluieren können.

II. **Zweck dieses Dokuments**
Dieses Dokument fasst alle funktionalen Anforderungen zusammen, definiert den Projektumfang und stellt die vorgeschlagene Verzeichnisstruktur für das Repository dar.

Zielsetzung:

I. **Primäres Ziel**
Bereitstellung eines stabilen Prototyps mit Grundfunktionen: Physiksimulation, Editiermodus, Puzzle- und Sandbox-Modus.

II. **Sekundäres Ziel**
Erweiterung des Prototyps durch optionale Features (Menüs, erweiterter Level-Editor, Grafikalternativen) zur Steigerung von Bedienkomfort, Wiederverwendbarkeit und visueller Identität.

Anforderungen:

Folgende Featuers müssen vollständig umgesetzt werden:

Pflichtfeatures:

I. **Low-Fidelity-Klon**
Reproduktion der Kernspielmechanik gemäß Kick-off-Video.

II. **Physiksimulation und Editiermodus**
Die zwei Hauptspielmodi, in denen das Spiel sich befinden kann. Dabei soll es möglich sein:

- zwischen Editiermodus (Objekte platzieren/entfernen) und Simulationsmodus (Play/-Stop) zu wechseln.
- mit Buttons zu navigieren: Start/Abbruch der Simulation, Zurücksetzen in den Editiermodus.

III. **Physikobjekte**
Folgende Arten an Objekten mit pysiksimulation sollten existieren:

- Statische und dynamische Objekte.
- Spezialobjekte (Antriebe, Sensoren) gemäß Kick-off-Folien.

IV. **Level-System und Puzzle-Modus**
Der Puzzelmodus soll mit folgenden Features erstellt werden:

- Levels als externe, menschenlesbare Dateien (JSON) speichern
- Import/Export und Verteilung via Dateien
- Automatische Level-Progression bei erfolgreichem Abschluss
- Siegesbedingungen, Einschränkungszonen, Inventarsystem für verfügbare Bauteile

V. **Sandboxmodus**:
Zum erleichtern der Erstellung des Puzzle-Modus wollen wir einen Spielplatz/Sandkiste (Sandbox) erstellen, die alle Objekte platzierbar macht und es ermöglicht eigene Level zu bauen.

- Leere Welt zum freien Bauen
- Import/Export von Levels im Sandbox-Kontext
- Optionale Puzzle-Features (Inventar, Einschränkungszonen) in der Sandbox

VI. **Grafik**
In der ersten Version genügen Primitive Formen als Objekte, wie Rechtecke oder Kreise, die in verschiedenen Farben angezeigt werden und mit einer Framerate ≥ 30 in echtzeit gerändert werden.

VII. **Persistenz**
Es soll dem Spieler per Import/Export von Level Dateien möglich sein, ein Level zu speichern und zu laden und der Spielzustand soll gespeichert werden.

Wahlfeatures:

Zusätzlich zu den Pflichtfeatures sollen drei Wahlfeatures aus 5 gewählt werden. Dabei wählten wir Features I Richtige Menüs, IV Level-Editor+ und V komplexere Graphiken.

I. **Menüs & Steuerung**
Hinzufügen von Menü Ansichten, die das Spielerlebnis verbessern:

- Titelbildschirm mit Start, Puzzle/Sandbox-Auswahl, Optionen (potentiell Audioeinstellungen, Texture-Pack auswählen)
- In-Game-Menü (Pause, Speichern, Beenden)

II. **Level-Editor+**
Wir erweiternden Editor, um die Art mit den Objekten im Editor zu interagieren zu verbessern.

- Drag & Drop um die Objekte zu platzieren und zu verschieben
- Verschieben und Rotieren von bereits platzierten Objekten
- Eine Undo/Redo-Funktion hinzufügen

III. **Komplexere Grafiken**
Wir wollen das Spiel anschaulicher darstellen mit erweiterten Grafikoptionen:

- Objekte als PNG-Texturen oder Sprites
- Austauschbare Texture-Packs
- Hintergrundgraphiken

Im weiteren Verlauf wird es weitere Features geben, für die wir auf Wunsch eingehen können müssen. Deshalb sollte weites gehend die Spielstruktur anpassbar sein.

Zusatzfeature:

Das per Change-Request angeforderte Feature kam nach der Präsentation des Minimum-Viable-Products. Das gewählte feature ist:

I. **Sandbox-Leveleditor mit Dateivorschau**

- Neue Level können mit Zugriff auf alle im Spiel enthaltenen Objekte und Features (z.B. Zielbedingungen) entweder aus einer leeren Szene oder basierend auf einer bestehenden Leveldatei erstellt werden.
- Die Simulation kann jederzeit als Testlauf gestartet werden (wie im Puzzle-Modus).
- Während des Editierens wird eine Vorschau der zugehörigen Leveldatei angezeigt.
- Änderungen im Leveleditor werden direkt in der Leveldateivorschau reflektiert.
- Änderungen in der angezeigten Leveldatei werden automatisch im Leveleditor übernommen.
- Der Nutzer erhält eine geeignete Fehlermeldung, wenn eine Änderung in der Datei nicht dem korrekten Format entspricht oder eine Platzierung unmöglich ist.

Storycards:

Um die gegebenen Anforderungen aus Nutzersicht besser zu verstehen, nutzen wir Storycards (Vorderseite | Rückseite):

Hoch | **Zwischen Bauen und Simulieren wechseln** | 05

Als Spieler möchte ich zwischen dem Bau-Modus und dem Spiel-Modus wechseln können, damit ich sehen kann, ob meine Konstruktion funktioniert.

Angenommen ich möchte meine Konstruktion testen, indem ich auf den Simulator - Modus Knopf drücke. Dann soll das Spiel mit meiner Konstruktion spielbar sein.

Mittel | 07

Hoch | **Puzzle-Modus mit Einschränkungen** | 10

Als Spieler möchte ich Levels mit festem Ziel und begrenztem Inventar spielen, um es wie ein echtes Rätsel zu nutzen.

Angenommen es gibt spielbare Level, wenn der Puzzel-Modus ausgewählt wird, dann öffne das vorgefertigte Level.

Hoch | 01

Mittel | **Spiel starten über den Startbildschirm** | 04

Als Spieler möchte ich über einen klaren Startbildschirm in verschiedene Modi wie Sandbox oder Puzzle wechseln können, um schnell mit dem Spielen oder Bauen zu beginnen.

Angenommen das Spiel ist gestartet, wenn ich auf „Start“, „Sandbox“ oder „Puzzles“ klicke, dann werde ich zum entsprechenden Modus weitergeleitet.

Niedrig | 03

Hoch | **Verschiedene Spielobjekte** | 03

Als Spieler möchte ich verschieden Objekte benutzen können, damit ich kreative Lösungen bauen kann.

Angenommen ich bin am erstellen eines Levels, dann möchte ich die Auswahl zwischen mehreren unterschiedlichen Objekten haben.

Mittel | 03

Hoch | **Level-Dateien speichern und Laden** | 07

Als Spieler möchte ich Level in einer Datei speichern oder aus ihnen laden, damit ich sie teilen oder damit später weitermachen kann.

Angenommen ich habe ein Level gebaut oder gelöst, wenn ich auf „Speichern“ oder „Exportieren“ klicke, dann wird mein Fortschritt gesichert. Wenn ich ein Level erhalte, dann möchte ich es laden können.

Mittel | 05

Hoch

Puzzle-Level auswählen

04

Als Puzzle-Spieler möchte ich Level mit Vorschau und Beschreibung auswählen können, um gezielt ein Level zu starten, das mich interessiert.

Angenommen ich befinde mich im Puzzle-Menü, dann sehe ich mehrere Level mit Thumbnail und Beschreibung. Wenn ich ein Level anklicke, dann kann ich das Level starten.

Mittel

01

Niedrig

Einstellungen ändern

02

Als Spieler möchte ich Texturpakete in den Optionen einstellen, um das Spiel an meine persönlichen Vorlieben anzupassen.

Angenommen ich befinde mich im Optionsmenü, wenn ich die Lautstärke oder das Texturepack auswähle, dann ändern sich die Spielausgabe und Darstellung entsprechend.

Niedrig

01

Hoch

Eigene Sandbox-Level bauen

08

Als kreativer Spieler möchte ich in einer leeren Welt eigene Level erstellen können, um meine eigenen Ideen frei umzusetzen.

Angenommen ich betrete den Sandbox-Modus, wenn ich Objekte platziere, dann entsteht mein eigenes, individuell gestaltetes Level.

Mittel

08

Niedrig

Level abschließen & weiterspielen

01

Als Spieler möchte ich nach einem erfolgreichen Level automatisch zum nächsten kommen, damit ich das nicht manuell machen muss.

Angenommen ich schließe ein Level ab, dann möchte ich auf einen Knopf drücken, der mich automatisch zum nächsten Level bringt.

Niedrig

01

Mittel

Besserer Editor

05

Als Spieler möchte ich Objekte mit der Maus verschieben und drehen, damit der Editor einfacher zu bedienen ist.

Angenommen der Leveleditor ist geöffnet, wenn ich ein Objekt auswähle, dann kann ich dieses rotieren, löschen oder verschieben

Hoch

02

Hoch **Einfache, aber klare Grafik** 01

Als Spieler möchte ich, dass Objekte klar unterschiedlich aussehen damit ich alles gut erkennen kann.

Angenommen ich spiele im Puzzel - Modus, dann möchte ich klar erkennen können, welche Objekte ich auswähle und platziere.

Mittel 01

Mittel **Puzzle-Regeln und Ziele sehen** 02

Als Puzzle-Spieler möchte ich die Siegbedingungen und Einschränkungen eines Levels erkennen, um das Levelziel effizient zu erreichen.

Angenommen ich betrete ein Puzzle-Level, dann sehe ich, was ich tun darf und was das Ziel ist.

Niedrig 01

Mittel **Undo/Redo im Editor nutzen** 05

Als Benutzer möchte ich Aktionen im Editor rückgängig machen oder wiederholen, um Fehler schnell zu korrigieren.

Angenommen ich habe eine Aktion im Editor gemacht, wenn ich auf „Undo“ oder „Redo“ klicke, dann wird die entsprechende Änderung zurückgenommen oder erneut angewendet.

Niedrig 01

Mittel **Schönere Grafiken** 04

Als Spieler möchte ich, dass das Spiel aus mehr als einfachen Graphiken besteht, um ein besseres Spielgefühl zu haben

Angenommen ich spiele das Spiel, wenn ich das Spiel anschaue, dann sehe ich mehr als einfache Formen.

Mittel 03

Hoch **Spielmenü** 04

Als Spieler möchte ich über ein Ingame-Menü das Spiel pausieren, speichern oder beenden können, um jederzeit Kontrolle über mein Spielerlebnis zu haben.

Angenommen ich befinde mich in einem aktiven Spiel, wenn ich die ESC-Taste drücke, dann öffnet sich ein Menü mit den Optionen „Speichern“, „Beenden“ und ggf. „Einstellungen“, die ich direkt nutzen kann.

Mittel 02

Luke Berger (770220)
Benedikt Schröder (773793)
Alexander Roskamp (777951)

Hoch

Bedienungsanleitung

01

Als spieler möchte ich eine Bedienungsanleitung haben, damit ich mich in dem System zurrecht finde.

Angenommen ich möchte mein Level speichern, aber ich weiß nicht wo ich das tue. Dann kann ich ganz einfach in der Bedienungsanleitung nachlesen und somit mein Level speichern!

Niedrig

01

Hoch

Text-Editor

03

Als User möchte ich einen Text Editor im Sandbox-Modus haben, um noch mehr Kontrolle beim bauen eigener Puzzles zu haben.

Angenommen ich möchte die Größe eines Balles ändern und ihn pixel genau an bestimmten Koordinaten platzieren. Wenn ich im Edior Koordinaten und Höhe und Breite ändere, dann ändert sich es auch in Echtzeit in der Vorschau.

Hoch

02

Hoch

Lesbare Leveldatei

04

Als Level-Designer möchte ich in der Lage sein die Leveldatei zu lesen – zu verstehen und dadurch leicht anpassen zu können.

Angenommen ich möchte wissen wie schwer ein Objekt ist, dann kann ich ganz einfach in der Leveldatei nachschauen und es intuitiv herauslesen.

Mittel

03

Anwendungsfalldiagramm:

Aus diesen Storycards und den oben gegebenen Features erstellten wir nun ein Anwendungsfalldiagramm:

Verzeichnisstruktur:

Die bisher aus den Anforderungen hervorgegangene Struktur haben wir in einem Ordnerbaum dargestellt:

```
project_root/
├── docs/
│   ├── design/
│   ├── sanity-checks/
│   │   ├── sanity_check_code/
│   │   ├── SanityCheck_JavaFX.pdf
│   │   └── SanityCheck_Jbox2d.pdf
│   ├── Anforderungsdokument.pdf
│   ├── Anforderungen.tex
│   ├── MadBalls_Features.pdf
│   ├── MadBalls_StoryCards.pdf
│   ├── MB_Gantt-Diagramm.png
│   └── OpenIssueReport.pdf
├── src/
│   ├── main/
│   │   ├── java/
│   │   │   ├── mm/
│   │   │   │   ├── controller/
│   │   │   │   ├── model/
│   │   │   │   └── view/
│   │   │   ├── module-info.java
│   │   │   └── Generated.java
│   │   └── resources/
│   │       ├── level/
│   │       ├── pictures/
│   │       └── styling/
│   └── test/
├── target/
├── .gitignore
├── CustomRules.xml
├── pom.xml
└── README.md
```

Architektur:

Wie in der Ordnerstruktur sichtbar folgt unser Projekt dem Model-View-Controller-Pattern, in dem in View-Directory alle View-Klassen sind, die Graphische Oberflächen erstellen. Im Model-Directory werden alle Klassen, die auf den Leveldaten arbeiten und/oder Simulationslogik definieren, erstellt. In Controller werden die einzelnen Model-Klassen und View-Klassen kontrolliert. Die Logik die die Models mit Views verknüpft und Datenfluss ermöglicht ist auch im Controller-Directory zu finden. Für eine Genauere Beschreibung schauen sie in der Architekturdokumentation nach.

Glossar:

- Low-Fidelity: *Funktionale, einfache Darstellung ohne aufwändige Grafiken.*
- Persistenz: *Speichern und Laden von Level-Zuständen.*
- Sandbox-Modus: *Freies Bauen ohne vorgegebene Puzzle-Ziele.*

- Inventarsystem: *Auswahlmenü für verfügbare Bauteile je Level.*

Luke Berger (770220)
Benedikt Schröder (773793)
Alexander Roskamp (777951)

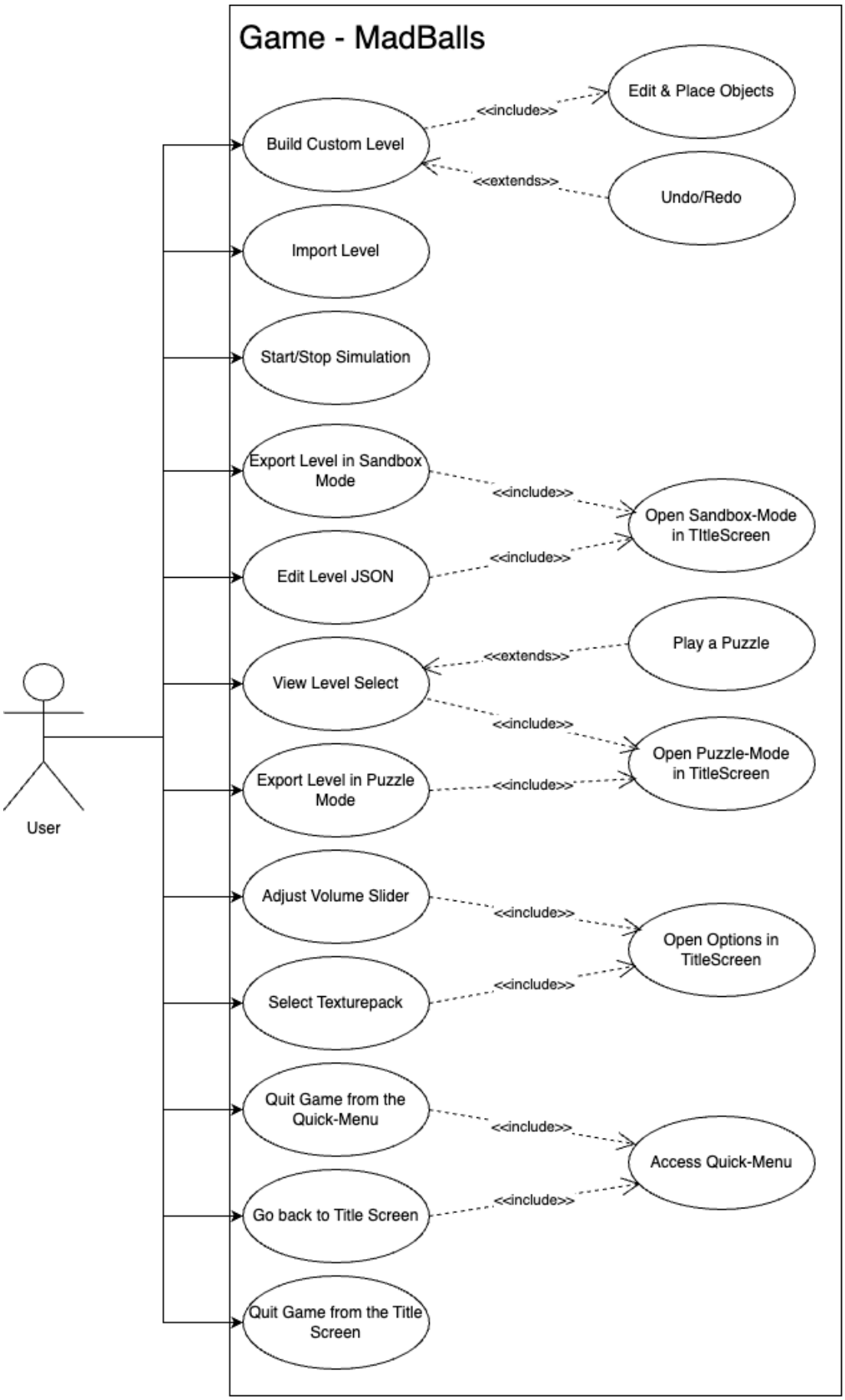

Abbildung 1: Anwendungsfalldiagramm